

```

import { useState, useEffect, useRef } from "react";

// — Fonts —————
const fontLink = document.createElement("link");
fontLink.rel = "stylesheet";
fontLink.href = "https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800&display=block";
document.head.appendChild(fontLink);

// — Palette —————
const C = {
  bg: "#0d0e12",
  surface: "#13151c",
  card: "#1a1d27",
  border: "#242736",
  amber: "#f5a623",
  amberDim: "#f5a62340",
  green: "#4ade80",
  greenDim: "#4ade8030",
  rose: "#fb7185",
  roseDim: "#fb718530",
  blue: "#60a5fa",
  blueDim: "#60a5fa30",
  purple: "#a78bfa",
  purpDim: "#a78bfa30",
  text: "#f0f0f5",
  muted: "#6b7280",
  faint: "#374151",
};

// — Global styles —————
const globalStyle = document.createElement("style");
globalStyle.textContent = `
  *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }
  body { background: ${C.bg}; color: ${C.text}; font-family: 'DM Sans', sans-serif; }
  ::-webkit-scrollbar { width: 4px; }
  ::-webkit-scrollbar-track { background: transparent; }
  ::-webkit-scrollbar-thumb { background: ${C.border}; border-radius: 2px; }
  @keyframes fadeUp { from { opacity: 0; transform: translateY(16px); } to { opacity: 1; transform: translateY(0); } }
  @keyframes pop { 0% { transform: scale(.85); opacity: 0; } 60% { transform: scale(1.1); opacity: 1; } }
  @keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: .45; } }
  @keyframes confetti { 0% { transform: translateY(0) rotate(0); } 100% { transform: translateY(100%) rotate(360deg); } }
  @keyframes spin { to { transform: rotate(360deg); } }
  @keyframes shimmer { 0% { background-position: -200% 0; } 100% { background-position: 0 0; } }
  @keyframes glow { 0%, 100% { box-shadow: 0 0 12px ${C.amber}55; } 50% { box-shadow: 0 0 12px ${C.amber}55; } }
`;

```

```

`;
document.head.appendChild(globalStyle);

// — Helpers —————
const uid = () => Math.random().toString(36).slice(2, 9);

const ENERGY_MAP = {
  low: { label: "Low", emoji: "🌊", color: C.blue, tasks: ["Reply emails", "U
  medium: { label: "Medium", emoji: "⚡", color: C.amber, tasks: ["Plan next week
  high: { label: "High", emoji: "🔥", color: C.rose, tasks: ["Record content",
};

const ROADMAPS = {
  "Clothing brand": ["Define your niche", "Choose brand name", "Check domain ava
  "AI tool": ["Validate problem", "Scope MVP features", "Choose tech stack
  "Food product": ["Develop recipe", "Cost per unit", "Find manufacturer", "Desi
  "Service business": ["Define offer clearly", "Price your service", "Build simple
  "Content brand": ["Pick one niche", "Choose primary platform", "Define content
};

const CATEGORIES = { idea: "💡 Idea", task: "✅ Task", problem: "⚠️ Problem", contei

// — Tiny components —————
const Pill = ({ label, color, dimColor }) => (
  <span style={{ background: dimColor, color, borderRadius: 20, padding: "2px 10p
    {label}
  </span>
);

const ProgressBar = ({ value, color, height = 6 }) => (
  <div style={{ background: C.border, borderRadius: height, overflow: "hidden", h
    <div style={{ width: `${Math.min(100, value)}%`, height: "100%", background:
  </div>
);

const Card = ({ children, style = {}, onClick }) => (
  <div onClick={onClick} style={{ background: C.card, border: `1px solid ${C.bord
    onMouseEnter={e => { if (onClick) { e.currentTarget.style.borderColor = C.amb
    onMouseLeave={e => { e.currentTarget.style.borderColor = C.border; e.currentT
    {children}
  </div>
);

const Btn = ({ children, color = C.amber, onClick, style = {}, dim }) => (
  <button onClick={onClick} style={{ background: dim ? `${color}20` : color, colo
    onMouseEnter={e => { e.currentTarget.style.opacity = ".85"; e.currentTarget.s
    onMouseLeave={e => { e.currentTarget.style.opacity = "1"; e.currentTarget.sty

```

```

    {children}
  </button>
);

// — Confetti —————
const Confetti = ({ active }) => {
  if (!active) return null;
  const pieces = Array.from({ length: 18 }, (_, i) => ({
    id: i, color: [C.amber, C.green, C.rose, C.blue, C.purple][i % 5],
    left: `${5 + i * 5.2}%`, delay: `${(i * 0.07).toFixed(2)}s`, size: 6 + (i % 4)
  }));
  return (
    <div style={{ position: "fixed", top: "40%", left: 0, width: "100%", pointerE
      {pieces.map(p => (
        <div key={p.id} style={{ position: "absolute", left: p.left, width: p.siz
        )}}
      </div>
    )};
  );
};

// =====
// SCREEN: Onboarding
// =====
const Onboarding = ({ onDone }) => {
  const [step, setStep] = useState(0);
  const [name, setName] = useState("");
  const [energy, setEnergy] = useState(null);
  const [build, setBuild] = useState(null);

  const steps = [
    {
      title: "Built for the way\nyou actually think.",
      sub: "Momentum is your operating system as a founder with a fire in your br
      content: (
        <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
          [{"No clutter", "No guilt", "No 50-feature overwhelm"].map(t => (
            <div key={t} style={{ display: "flex", alignItems: "center", gap: 10,
              <span style={{ color: C.green, fontSize: 18 }}>✓</span> {t}
            </div>
          )}}
        </div>
      ),
    },
    {
      title: "What should we\ncall you?",
      sub: "First name is fine.",
      content: (

```

```

      <input value={name} onChange={e => setName(e.target.value)} placeholder="
        style={{ width: "100%", background: C.surface, border: `1px solid ${C.b
        onFocus={e => e.target.style.borderColor = C.amber}
        onBlur={e => e.target.style.borderColor = C.border}
      />
    ),
  },
  {
    title: "What are you\nbuilding right now?",
    sub: "Pick the closest one.",
    content: (
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 10 }}
        {Object.keys(ROADMAPS).map(k => (
          <div key={k} onClick={() => setBuild(k)} style={{ background: build =
            {k}
          </div>
        ))}
      </div>
    ),
  },
  {
    title: "How's your energy\nright now?",
    sub: "Be honest. The app will adapt.",
    content: (
      <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
        {Object.entries(ENERGY_MAP).map(([k, v]) => (
          <div key={k} onClick={() => setEnergy(k)} style={{ background: energy
            <span style={{ fontSize: 24 }}>{v.emoji}</span>
          <div>
            <div style={{ fontWeight: 700, fontSize: 15, color: energy === k
              <div style={{ fontSize: 12, color: C.muted, marginTop: 2 }}>{v.ta
            </div>
          </div>
        ))}
      </div>
    ),
  },
];

```

```

const canNext = step === 1 ? name.trim().length > 1 : step === 2 ? !!build : st

```

```

return (
  <div style={{ minHeight: "100vh", display: "flex", flexDirection: "column", j
    /* Logo */
    <div style={{ marginBottom: 40, animation: "fadeUp .5s ease both" }}>
      <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 22, fontWeight:
    </div>

```

```

    {/* Step indicator */}
    <div style={{ display: "flex", gap: 6, marginBottom: 32, animation: "fadeUp
      {steps.map((_, i) => (
        <div key={i} style={{ height: 3, flex: 1, borderRadius: 2, background:
          )}}
      </div>

    {/* Content */}
    <div key={step} style={{ animation: "fadeUp .4s ease both" }}>
      <h1 style={{ fontFamily: "'Syne', sans-serif", fontSize: 30, fontWeight:
        <p style={{ color: C.muted, fontSize: 15, marginBottom: 28 }}>{steps[step]
          {steps[step].content}
        </div>

    {/* Nav */}
    <div style={{ marginTop: 32, display: "flex", gap: 12, animation: "fadeUp .
      {step > 0 && <Btn color={C.muted} dim onClick={() => setStep(s => s - 1)}
      <Btn onClick={() => step < steps.length - 1 ? setStep(s => s + 1) : onDon
        style={{ flex: 2, opacity: canNext ? 1 : .4, pointerEvents: canNext ? "
          {step === steps.length - 1 ? "Let's build 🔥" : "Continue →"}
        </Btn>
      </div>
    </div>
  );
};

// =====
// SCREEN: Focus Timer
// =====

const FocusTimer = ({ onClose }) => {
  const TOTAL = 20 * 60;
  const [seconds, setSeconds] = useState(TOTAL);
  const [running, setRunning] = useState(false);
  const [done, setDone] = useState(false);
  const ref = useRef(null);

  useEffect(() => {
    if (running && seconds > 0) {
      ref.current = setInterval(() => setSeconds(s => { if (s <= 1) { setRunning(
        }
      return () => clearInterval(ref.current);
    }, [running]);

  const pct = ((TOTAL - seconds) / TOTAL) * 100;
  const mm = String(Math.floor(seconds / 60)).padStart(2, "0");
  const ss = String(seconds % 60).padStart(2, "0");

```

```

const r = 90, circ = 2 * Math.PI * r;

return (
  <div style={{ position: "fixed", inset: 0, background: C.bg, zIndex: 100, display: "flex", flexDirection: "column", align-items: "center", justify-content: "center", height: "100%", width: "100%" }}>
    <button onClick={onClose} style={{ position: "absolute", top: 24, left: 24, width: 40px, height: 40px, border: 1px solid C.border, border-radius: 50%, background: "white", opacity: 0.5 }}>✕</button>

    <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 13, fontWeight: 700, text-align: "center", margin: 24px 0 }}>
      <div style={{ position: "relative", width: 220px, height: 220px, margin: 0 auto 20px auto }}>
        <svg width={220} height={220} style={{ transform: "rotate(-90deg)" }}>
          <circle cx={110} cy={110} r={r} fill="none" stroke={C.border} strokeWid=
          <circle cx={110} cy={110} r={r} fill="none" stroke={done ? C.green : C.muted} strokeDasharray={circ} strokeDashoffset={circ - (circ * pct / 100)} strokeLinecap="round" style={{ transition: "stroke-dashoffset 1s linear" }}>
        </svg>
        <div style={{ position: "absolute", inset: 0, display: "flex", flexDirection: "column", align-items: "center", justify-content: "center", height: "100%", width: "100%" }}>
          {done ? (
            <div style={{ fontSize: 48, animation: "pop .5s ease" }}>🎉</div>
          ) : (
            <>
              <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 42, fontW
              <div style={{ color: C.muted, fontSize: 13, marginTop: 4 }}>minutes
            </div>
          </div>
        </div>
      </div>

      {done ? (
        <div style={{ textAlign: "center", animation: "fadeUp .4s ease" }}>
          <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 24, fontWeigh
          <div style={{ color: C.muted, marginBottom: 32 }}>You stayed locked in.
          <Btn onClick={onClose} color={C.green}>Back to dashboard</Btn>
        </div>
      ) : (
        <div style={{ display: "flex", gap: 12 }}>
          <Btn onClick={() => setRunning(r => !r)} style={{ minWidth: 140 }}>
            {running ? "⏸ Pause" : seconds === TOTAL ? "▶ Start Sprint" : "▶ Res
          </Btn>
          {seconds < TOTAL && <Btn color={C.muted} dim onClick={() => { setSecond
        </div>
      </div>
    )}

    {running && (
      <div style={{ marginTop: 32, color: C.muted, fontSize: 13, textAlign: "ce
        🎧 Stay off your phone. You've got this.
      </div>
    )}
  </div>

```

```

    })
  </div>
);
};

// =====
// SCREEN: Brain Dump
// =====

const BrainDump = ({ dumps, onAdd, onClose }) => {
  const [text, setText] = useState("");
  const [cat, setCat] = useState("idea");

  const autoDetect = (t) => {
    const lower = t.toLowerCase();
    if (/problem|issue|fix|broken|bug/.test(lower)) return "problem";
    if (/post|video|tiktok|reel|content|caption/.test(lower)) return "content";
    if (/email|invoice|admin|meeting|call/.test(lower)) return "admin";
    if (/build|create|launch|ship|make/.test(lower)) return "task";
    return "idea";
  };

  const handleChange = (v) => { setText(v); setCat(autoDetect(v)); };

  const submit = () => {
    if (!text.trim()) return;
    onAdd({ id: uid(), text, cat, ts: Date.now() });
    setText(""); setCat("idea");
  };

  return (
    <div style={{ position: "fixed", inset: 0, background: C.bg, zIndex: 100, display: "flex", alignItems: "center", gap: 12, marginBottom: 100 }}>
      <button onClick={onClose} style={{ background: "none", border: "none", color: "white", padding: 10 20 }}>
        <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 20, fontWeight: "bold" }}>X</div>
      </button>

      <div style={{ flex: 1, padding: 10 20 10 100 }}>
        <div style={{ display: "flex", gap: 10 }}>
          <div style={{ flex: 1, padding: 10 0 10 10 }}>
            <div style={{ display: "flex", gap: 6 }}>
              <div style={{ display: "flex", gap: 6 }}>
                {Object.entries(CATEGORIES).map(([k, v]) => (
                  <button key={k} onClick={() => setCat(k)} style={{ background: cat === k ? C.amberDim : "transparent", border: 1px solid C.border, padding: 5 10 }}>
                    {v.split(" ")[0]}
                  </button>
                ))}
              <div style={{ flex: 1, padding: 5 10 }}>
                <input type="text" value={text} onChange={e => handleChange(e.target.value)} style={{ width: 100%, background: "none", border: "none", outline: "none" }}>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
};

```

```

        )))
      </div>
      <Btn onClick={submit} style={{ padding: "8px 16px", fontSize: 13 }}>Dum
    </div>
  </div>

  { /* Auto-detect hint */ }
  <div style={{ color: C.muted, fontSize: 12, marginBottom: 20 }}>
    Auto-detected: <span style={{ color: C.amber }}>{CATEGORIES[cat]}{d.text
      </div>
    ))}
  </div>
</div>
);
};

// =====
// SCREEN: Not Now (Parking Lot)
// =====

const NotNow = ({ ideas, onAdd, onClose }) => {
  const [text, setText] = useState("");

  return (
    <div style={{ position: "fixed", inset: 0, background: C.bg, zIndex: 100, dis
      <div style={{ display: "flex", alignItems: "center", gap: 12, marginBottom:
        <button onClick={onClose} style={{ background: "none", border: "none", co
        <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 20, fontWeight:
      </div>
    <p style={{ color: C.muted, fontSize: 13, marginBottom: 24, paddingLeft: 36

    <div style={{ display: "flex", gap: 10, marginBottom: 20 }}>
      <input value={text} onChange={e => setText(e.target.value)} placeholder="
        onKeyDown={e => { if (e.key === "Enter" && text.trim()) { onAdd(text.tr
          style={{ flex: 1, background: C.surface, border: `1px solid ${C.border}
          onFocus={e => e.target.style.borderColor = C.purple}
          onBlur={e => e.target.style.borderColor = C.border} />
      <Btn color={C.purple} onClick={() => { if (text.trim()) { onAdd(text.trim
    </div>

```

```

    <div style={{ flex: 1, overflowY: "auto", display: "flex", flexDirection: "
      {ideas.length === 0 && <div style={{ color: C.muted, textAlign: "center",
        {ideas.map((idea, i) => (
          <div key={i} style={{ background: C.card, border: `1px solid ${C.border
            <span style={{ fontSize: 18 }}>🚀</span>
            <span style={{ fontSize: 14, flex: 1 }}>{idea}</span>
            <span style={{ fontSize: 11, color: C.muted }}>parked</span>
          </div>
        )}}
      </div>
    </div>
  );
};

// =====
// SCREEN: Roadmap
// =====
const Roadmap = ({ build, onClose }) => {
  const steps = ROADMAPS[build] || [];
  const [done, setDone] = useState({});
  const completedCount = Object.values(done).filter(Boolean).length;

  return (
    <div style={{ position: "fixed", inset: 0, background: C.bg, zIndex: 100, dis
      <div style={{ display: "flex", alignItems: "center", gap: 12, marginBottom:
        <button onClick={onClose} style={{ background: "none", border: "none", co
          <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 20, fontWeight:
        </div>
        <div style={{ color: C.amber, fontSize: 13, fontWeight: 700, marginBottom:

        <ProgressBar value={(completedCount / steps.length) * 100} color={C.amber}
        <div style={{ color: C.muted, fontSize: 12, textAlign: "right", marginTop:

        <div style={{ flex: 1, overflowY: "auto", display: "flex", flexDirection: "
          {steps.map((s, i) => {
            const isDone = done[i];
            const isNext = !isDone && Object.values(done).filter(Boolean).length ==
            return (
              <div key={i} onClick={() => setDone(d => ({ ...d, [i]: !d[i] })))}
                style={{ background: isDone ? C.greenDim : isNext ? C.amberDim : C.
              <div style={{ width: 28, height: 28, borderRadius: "50%", backgroun
                {isDone ? "✓" : i + 1}
              </div>
              <div style={{ flex: 1 }}>
                <div style={{ fontSize: 14, fontWeight: isDone || isNext ? 700 :
                  {isNext && <div style={{ fontSize: 11, color: C.amber, marginTop:

```

```

        </div>
    </div>
    );
  }}}
</div>
</div>
);
};

// =====
// SCREEN: Dashboard
// =====

const Dashboard = ({ user, onReset }) => {
  const [wins, setWins] = useState([
    { id: uid(), text: "Write today's content script", done: false },
    { id: uid(), text: "Follow up with 3 prospects", done: false },
    { id: uid(), text: "Review pricing page", done: false },
  ]);
  const [screen, setScreen] = useState(null); // "focus" | "dump" | "notnow" | "r
  const [dumps, setDumps] = useState([]);
  const [parked, setParked] = useState([]);
  const [confetti, setConfetti] = useState(false);
  const [streak] = useState(4);
  const [momentumScore] = useState(72);
  const [editingWin, setEditingWin] = useState(null);
  const [newWinText, setNewWinText] = useState("");

  const energy = ENERGY_MAP[user.energy];
  const doneCount = wins.filter(w => w.done).length;

  const toggleWin = (id) => {
    setWins(ws => ws.map(w => w.id === id ? { ...w, done: !w.done } : w));
    const isCompleting = !wins.find(w => w.id === id)?.done;
    if (isCompleting) {
      setConfetti(true);
      setTimeout(() => setConfetti(false), 1200);
    }
    const newDone = wins.filter(w => w.done).length + (isCompleting ? 1 : -1);
    if (newDone === 3 && isCompleting) {
      setTimeout(() => { setConfetti(true); setTimeout(() => setConfetti(false),
    }
  };

  if (screen === "focus") return <FocusTimer onClose={() => setScreen(null)} />;
  if (screen === "dump") return <BrainDump dumps={dumps} onAdd={d => setDumps(ds
  if (screen === "notnow") return <NotNow ideas={parked} onAdd={t => setParked(p
  if (screen === "roadmap") return <Roadmap build={user.build} onClose={() => set

```

```

return (
  <div style={{ maxWidth: 420, margin: "0 auto", padding: "24px 18px 100px", mi
    <Confetti active={confetti} />

    {/* Header */}
    <div style={{ display: "flex", justifyContent: "space-between", alignItems:
      <div>
        <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 13, fontWeigh
          <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 26, fontWeigh
        </div>
        <div style={{ textAlign: "right" }}>
          <div style={{ background: `${energy.color}20`, border: `1px solid ${ene
            <span style={{ fontSize: 16 }}>{energy.emoji}</span>
            <span style={{ fontSize: 12, fontWeight: 700, color: energy.color }}>
          </div>
          <div style={{ fontSize: 11, color: C.muted, marginTop: 4 }}>🔥 {streak}
        </div>
      </div>

      {/* Momentum score */}
      <Card style={{ marginBottom: 16, animation: "fadeUp .4s .05s ease both" }}>
        <div style={{ display: "flex", justifyContent: "space-between", alignItem
          <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 13, fontWeigh
            <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 22, fontWeigh
          </div>
          <ProgressBar value={momentumScore} color={C.amber} height={8} />
          <div style={{ display: "flex", justifyContent: "space-between", marginTop
            <span style={{ fontSize: 11, color: C.muted }}>Execution Level: <span s
              <span style={{ fontSize: 11, color: C.muted }}>Next: <span style={{ col
            </div>
          </Card>

      {/* Energy suggestion */}
      <div style={{ background: `${energy.color}12`, border: `1px solid ${energy.
        <div style={{ fontSize: 12, color: energy.color, fontWeight: 700, marginB
          <div style={{ fontSize: 13, color: C.text }}>{energy.tasks.slice(0, 3).jo
        </div>

      {/* 3 Wins */}
      <div style={{ marginBottom: 20, animation: "fadeUp .4s .1s ease both" }}>
        <div style={{ display: "flex", justifyContent: "space-between", alignItem
          <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 16, fontWeigh
            < pill label={` ${doneCount}/3` } color={doneCount === 3 ? C.green : C.amb
          </div>
          <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
            {wins.map((w, i) => (

```

```

<div key={w.id}
  style={{ background: w.done ? C.greenDim : C.card, border: `1.5px s
  onClick={() => { if (editingWin !== w.id) toggleWin(w.id); }}>
  <div style={{ width: 26, height: 26, borderRadius: "50%", border: `
    {w.done ? "✓" : ""}
  </div>
  {editingWin === w.id ? (
    <input autoFocus defaultValue={w.text}
      onBlur={e => { setWins(ws => ws.map(x => x.id === w.id ? { ...x
      onKeyDown={e => { if (e.key === "Enter") e.target.blur(); }}
      style={{ flex: 1, background: "none", border: "none", outline:
    ) : (
      <span style={{ flex: 1, fontSize: 14, fontWeight: 500, textDecora
      onDoubleClick={() => setEditingWin(w.id)}>{w.text}</span>
    )}
    {!w.done && <span style={{ fontSize: 11, color: C.faint }}></span>
  </div>
  )}}
</div>
{doneCount === 3 && (
  <div style={{ textAlign: "center", marginTop: 16, animation: "pop .5s e
    <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 16, fontWei
  </div>
  )}
</div>

{/* Focus Button */}
<div style={{ marginBottom: 16, animation: "fadeUp .4s .22s ease both" }}>
  <button onClick={() => setScreen("focus")}
    style={{ width: "100%", background: `linear-gradient(135deg, ${C.amber}
    <div style={{ textAlign: "left" }}>
      <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 18, fontWei
      <div style={{ fontSize: 12, color: `${C.bg}99`, marginTop: 2 }}>20 mi
    </div>
      <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 32, fontWeigh
    </button>
  </div>

{/* Brain dump + Not Now */}
<div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 12, mar
  <button onClick={() => setScreen("dump")}
    style={{ background: C.card, border: `1px solid ${C.border}`, borderRad
    onMouseEnter={e => { e.currentTarget.style.borderColor = C.blue; }}
    onMouseLeave={e => { e.currentTarget.style.borderColor = C.border; }}>
    <div style={{ fontSize: 24, marginBottom: 6 }}>⚡</div>
    <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 14, fontWeigh
    <div style={{ fontSize: 11, color: C.muted, marginTop: 2 }}>{dumps.leng

```

```

    </button>
    <button onClick={() => setScreen("notnow")}
      style={{ background: C.card, border: `1px solid ${C.border}`, borderRadius
      onMouseEnter={e => { e.currentTarget.style.borderColor = C.purple; }}
      onMouseLeave={e => { e.currentTarget.style.borderColor = C.border; }}>
      <div style={{ fontSize: 24, marginBottom: 6 }}><img alt="rocket icon" data-bbox="660 140 685 160"/></div>
      <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 14, fontWeigh
      <div style={{ fontSize: 11, color: C.muted, marginTop: 2 }}>{parked.len
    </button>
  </div>

  { /* Roadmap */ }
  <Card onClick={() => setScreen("roadmap")} style={{ marginBottom: 20, anima
    <div style={{ display: "flex", justifyContent: "space-between", alignItem
      <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 14, fontWeigh
        < Pill label={user.build} color={C.amber} dimColor={C.amberDim} />
      </div>
      <ProgressBar value={14} color={C.green} height={6} />
      <div style={{ fontSize: 12, color: C.muted, marginTop: 8 }}>1/{ROADMAPS[u
    </Card>

  { /* Stats row */ }
  <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", gap: 10,
    {[
      { label: "Focus hrs", value: "3.2", color: C.amber },
      { label: "Tasks done", value: `${doneCount * 4 + 11}`, color: C.green }
      { label: "Ideas parked", value: `${parked.length}`, color: C.purple },
    ].map(s => (
      <Card key={s.label} style={{ textAlign: "center", padding: "14px 10px"
        <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 22, fontWei
          <div style={{ fontSize: 10, color: C.muted, marginTop: 2, textTransfo
        </Card>
      )))
    </div>

  { /* Reset */ }
  <div style={{ textAlign: "center", marginTop: 32 }}>
    <button onClick={onReset} style={{ background: "none", border: "none", co
  </div>
</div>

);
};

// =====
// ROOT
// =====
export default function App() {

```

```
const [user, setUser] = useState(null);

return user
  ? <Dashboard user={user} onReset={() => setUser(null)} />
  : <Onboarding onDone={setUser} />;
}
```